

Лабораторная работа №5

«Интеграция обученной модели нейроклассификатора в веб-приложение Flask»

Цель работы: Обучить модель искусственной нейронной сети, и интегрировать ее в веб-приложение. Реализовать отправку изображения на сервер и обработку изображения моделью.

Ход работы: Создайте пакет «model», в котором создайте файл «nn.py», в нем будет находиться программный код реализации архитектуры модели, которую Вы обучали для классификации типов фигур. В этом же пакете будет находиться файл с оптимальными, найденными в процессе обучения модели весами искусственной нейронной сети. Пример модели представлен на рисунке 1. Конечно, предварительно должны быть установлены необходимые фреймворки и библиотеки для реализации данного кода. При чем той же самой версии, чтобы не было ошибок.

```
from keras.models import Sequential
from keras.layers import Dense
import numpy as np
from PIL import Image
import io
import os
# Определяем глобальную переменную ДО функций
_model = None
def load_model():
    # Создание последовательной модели
    model = Sequential()
    model.add(Dense(3000, input_dim=400, activation='linear',
name='Dense_5000'))
    model.add(Dense(1000, activation='relu', name='Dense_100'))
    model.add(Dense(200, activation='relu', name='Dense_1000'))
    model.add(Dense(10, activation='linear', name='Dense_10'))
    model.add(Dense(3, activation='softmax', name='Dense_3'))
    # Получаем путь к папке, где находится этот файл (nn.py)
    current_dir = os.path.dirname(os.path.abspath(__file__))
    # Путь к файлу весов в той же папке
    weights_path = os.path.join(current_dir, "best.weights.h5")
    model.load_weights(weights_path)
    return model
def get_model():
    #Возвращает загруженную модель (загружает, если ещё не загружена)
    global _model
    if _model is None:
        _model = load_model()
    return _model
```

Рисунок 1. – Одна из возможных реализаций модели нейроклассификатора

Следует отметить, что данная модель и данная архитектура не является оптимальной. Вы можете предложить и обучить свою модель.

Следующим этапом необходимо модифицировать контроллер, который реализован в файле «app.py». Он содержит всего один эндпоинт. В нем будет реализован функционал получения цифрового изображения и дальнейшая его обработка, включая нормализацию, а также использование обученной модели для классификации. Пример кода приведен на рисунке 2.

```
from flask import Flask, render_template, request, jsonify
from model import nn
import numpy as np
import io
```

```

from PIL import Image

app = Flask(__name__)

# Загружаем модель при старте
model = nn.get_model()

@app.route('/', methods=['GET', 'POST'])
def index():
    if request.method == 'POST':
        try:
            # Проверяем, есть ли файл в запросе
            if 'file' not in request.files:
                return jsonify({'error': 'Файл не найден'}), 400

            file = request.files['file']

            # Проверяем, выбран ли файл
            if file.filename == '':
                return jsonify({'error': 'Файл не выбран'}), 400

            # Читаем файл
            image_bytes = file.read()

            # Открываем изображение
            img = Image.open(io.BytesIO(image_bytes)).convert('L')
            img = img.resize((20, 20))

            # Предобработка
            #img_array = np.array(img) / 255.0
            # Предобработка учитывающая инверсию цвета:
            img_array = 1.0 - (np.array(img) / 255.0)
            img_flattened = img_array.flatten().reshape(1, -1)

            # Предсказание
            predictions = model.predict(img_flattened, verbose=0)[0]

            # Определение класса
            classes = ['triangle', 'circle', 'square']
            class_index = np.argmax(predictions)
            confidence = float(predictions[class_index])
            predicted_class = classes[class_index]

            # Возвращаем JSON для JavaScript
            return jsonify({
                'class': predicted_class,
                'confidence': confidence
            })

        except Exception as e:
            return jsonify({'error': str(e)}), 500

    # GET запрос - показываем страницу
    return render_template('index.html')

if __name__ == '__main__':
    app.run(debug=True)

```

Рисунок 2- Реализация контроллера

Обратите внимание, что при предобработке изображения применяется инверсия пикселей (рисунок 3). Предобработка должна включать нормализацию. Для изображений она заключается в делении пикселей на 255, то есть `img_array = np.array(img) / 255.0`. Однако, если мы загружаем геометрические фигуры черного цвета на белом фоне, то модель не будет работать корректно. Связано это с тем, что модель обучалась на наборе данных, представляющих собой фигуры белого цвета на черном фоне. Поэтому, предобработка изображений должна включать инверсию пикселей, так она достигается `img_array = 1.0 - (np.array(img) / 255.0)`.

Теперь необходимо модифицировать HTML разметку, добавив в тег формы атрибут, реализующий отправку цифрового изображения на сервер с POST запросом. Для этого модифицируйте код в шаблоне “index.html” связанный с формой следующим образом:

```
<form id="uploadForm" method="POST" action="/" class="upload-form"
enctype="multipart/form-data">
  <div class="upload-area" id="uploadArea">
    <div class="upload-content">
      <svg class="upload-icon" width="48" height="48" viewBox="0 0 24
24" fill="none" stroke="currentColor" stroke-width="2">
        <path d="M21 15v4a2 2 0 0 1-2 2H5a2 2 0 0 1-2-2v-4" />
        <polyline points="17 8 12 3 7 8" />
        <line x1="12" y1="3" x2="12" y2="15" />
      </svg>
      <h3>Перетащите изображение сюда</h3>
      <p>или</p>
      <button type="button" class="btn btn-primary"
id="browseBtn">Выберите файл</button>
      <input type="file" id="fileInput" name="file" accept="image/*"
hidden>
      <p class="file-info">Поддерживаются: JPG, PNG, JPEG (макс.
10MB)</p>
    </div>
    <div class="upload-preview" id="uploadPreview" style="display:
none;">
      <img id="previewImage" src="" alt="Preview">
      <button type="button" class="btn btn-remove"
id="removeImage">×</button>
    </div>
  </div>

  <div class="submit-section">
    <button type="submit" class="btn btn-submit" id="submitBtn" disabled>
      <span class="btn-text">Распознать фигуру</span>
      <span class="btn-loader" style="display: none;">⌛</span>
    </button>
  </div>
</form>
```

Рисунок 3. – Добавление атрибутов для отправки изображения на сервер

Атрибуты `method="POST"` `action="/"` указывают, что мы отправляем изображение с POST запросом эндпоинт ("/"). Также необходимо модифицировать JavaScript код, реализующий fetch API. В первом аргументе функции fetch замените `'/predict'` на `'/'`, как показано на рисунке 4.

```
try {

  const response = await fetch('/', {
    method: 'POST',
    body: formData
  });
```

```
const data = await response.json();

if (response.ok) {
  // Показываем результат
  displayResult(data);
} else {
  showError(data.error || 'Произошла ошибка при обработке изображения');
}
```

Рисунок 4. – Модификация fetch.

После запуска приложения, и отправки изображения на сервер должно появиться сообщение о результатах классификации объекта. Пример приведен на рисунке 5.

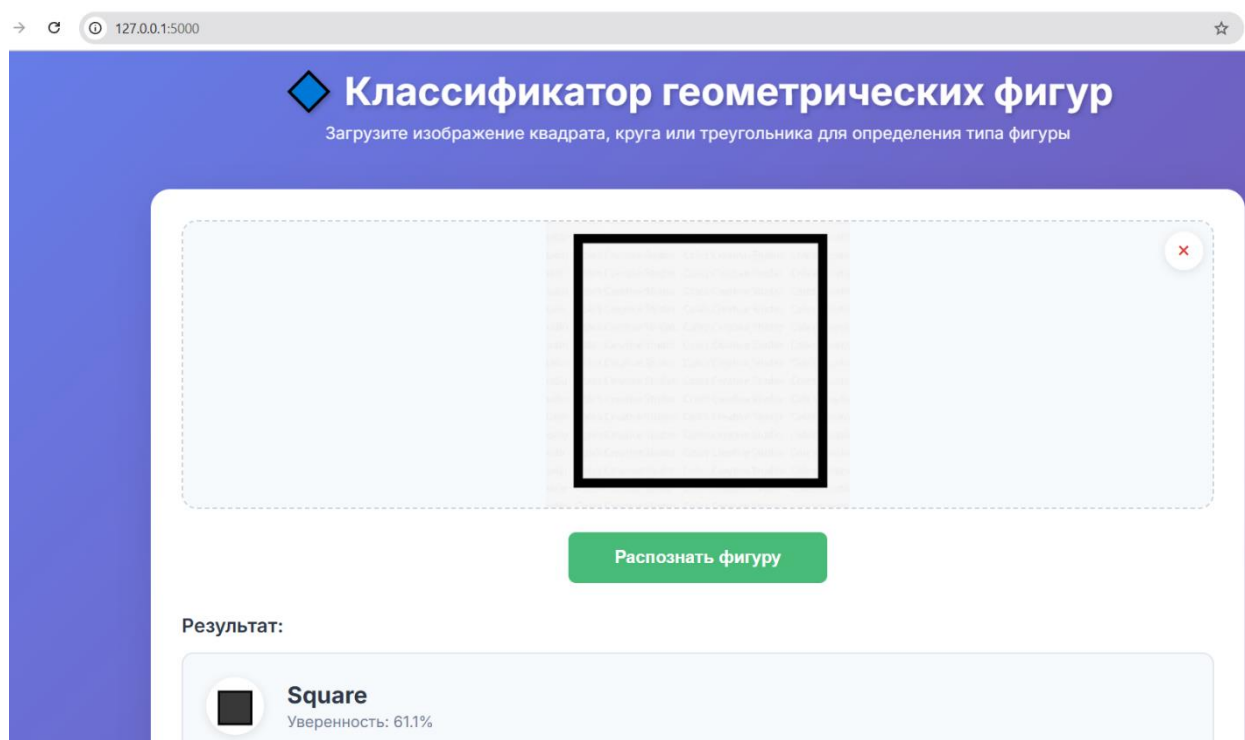


Рисунок 5. – Результат работы нейроклассификатора

В данной реализации алгоритм предобработки цифрового изображения реализован в соответствующем эндпоинте, то есть в контроллере.

Реализуйте алгоритм предобработки в файле “predictor.py”, который находится в пакете “service”. В контроллере вызывайте соответствующую функцию, которая будет реализовывать необходимую предобработку.

В качестве дополнительного задания реализуйте нормализацию пикселей, предусматривающая возможность работы модели как с изображениями геометрических фигур белых на черном фоне, так и черных на белом фоне. То есть, сделайте нейроклассификатор более универсальным.

Подумайте, какие более интересные задачи можно решить, используя описанный подход, а именно, какие объекты мы можем классифицировать на цифровых изображениях, и где данная информация затем может быть использована.